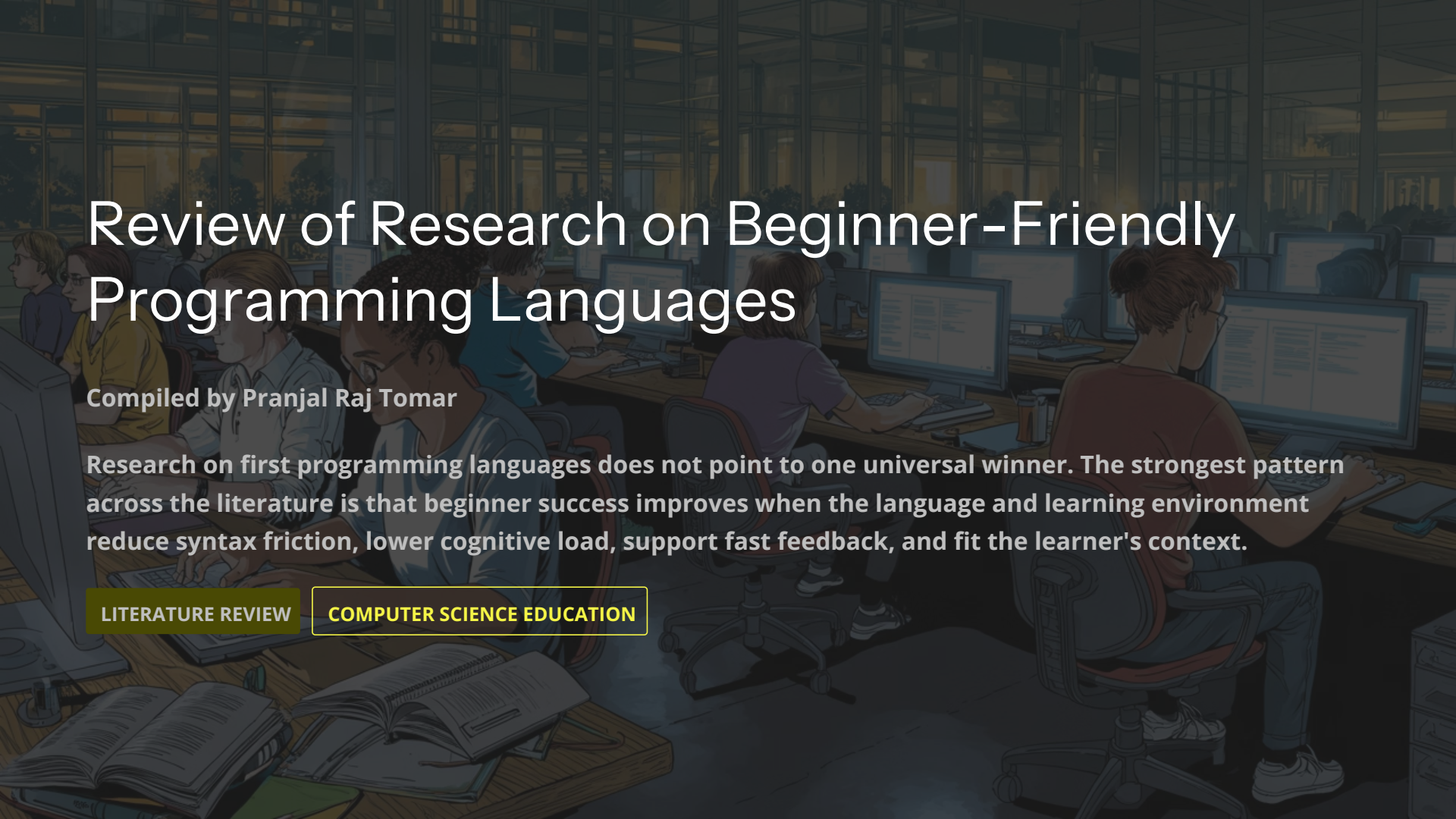


An illustration of a computer lab or classroom at night. Several students are seated at long wooden desks, each with a computer monitor. The students are focused on their work, with some looking at the screens and others at books or papers. The room has large windows in the background, showing a dark cityscape. The overall atmosphere is one of quiet concentration and learning.

Review of Research on Beginner-Friendly Programming Languages

Compiled by Pranjal Raj Tomar

Research on first programming languages does not point to one universal winner. The strongest pattern across the literature is that beginner success improves when the language and learning environment reduce syntax friction, lower cognitive load, support fast feedback, and fit the learner's context.

LITERATURE REVIEW

COMPUTER SCIENCE EDUCATION

What the Research Says

Across decades of scholarship, a consistent theme emerges: the best first language is not defined by industry popularity, but by how well it supports novice cognition and sustained engagement. This review synthesizes ten key studies spanning from 1977 to 2025, covering language design theory, cognitive load measurement, empirical comparisons, and learner psychology.

Reduce Syntax Friction

Minimize barriers between thought and code execution

Lower Cognitive Load

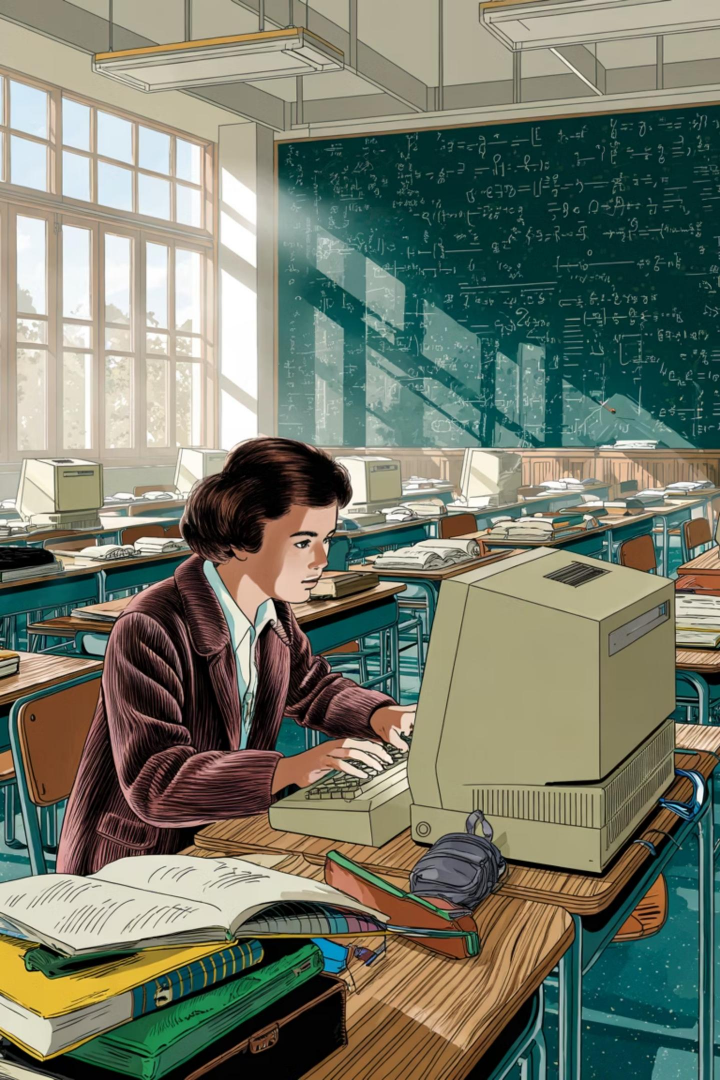
Design for working memory constraints of novices

Fast Feedback

Immediate, meaningful responses to learner actions

Context Fit

Align language choice with course goals and learner background



Early Teaching Should Prioritize Pedagogy Over Industry Status

FINDING 1

Robert P. Taylor's landmark 1977 paper challenged the assumption that widely used languages like BASIC, FORTRAN, and COBOL were suitable for teaching structured programming to raw beginners. Taylor argued that these languages, while dominant in industry, carried structural and syntactic baggage that actively hindered the development of sound programming habits in novices.

Taylor's Core Argument

Industry relevance does not equal pedagogical suitability. A first language should be designed specifically for learning — not adapted from professional use cases.

Why It Still Matters

Taylor's insight remains foundational: educators must resist choosing languages based on prestige or market share alone. Pedagogical fit must come first.

Blocks and Visual Abstractions Help Beginners

FINDING 2

Visual programming environments have transformed introductory computing by reducing the cognitive overhead associated with textual syntax. Two complementary approaches have emerged as especially influential

Block-Based Programming

Bau, Gray, Kelleher, Sheldon, and Turbak demonstrate that block-based systems reduce cognitive load, favor recognition over recall, and prevent syntax errors — making them foundational in introductory computing education.

Frame-Based Editing

Kölling, Brown, and Altadmri propose frame-based editing as a bridge between block-based and text-based programming. This hybrid approach preserves the scaffolding benefits of visual tools while gradually introducing textual syntax, serving as a stepping stone toward professional languages.

Cognitive Load Is a Central Issue in Beginner Programming

FINDING 3

Language characteristics directly impact the mental effort required of novice programmers. When cognitive load is too high, learners abandon the task before developing competence.

Language Design for Reduced Burden

Winkler and Flatscher argue that language characteristics should be chosen explicitly to reduce the burden on novices. They present REXX as a language supporting fast learning and frustration-free productivity — a model for how design choices can ease the beginner experience.

Measuring Cognitive Load

Zavgorodniaia, Duran, Hellas, Seppälä, and Sorva show that cognitive load in programming can be measured meaningfully through replication studies. However, they caution that the construct still requires more rigorous research before it can be fully operationalized in curriculum design.

Educational Language Design Still Matters

FINDING 4

Michael Kölling's 2024 paper reaffirms that principled language design is not a relic of early computing — it remains essential for effective pedagogy. While no permanent consensus exists (because teaching contexts and learner needs keep changing), a core set of design principles has endured.

1

Simplicity

Minimize language features that do not serve learning goals

2

Modularity

Support clear, independent units of code that mirror conceptual structure

3

Orthogonality

Ensure language features combine predictably without surprising interactions

4

Readability

Code should communicate intent clearly to both humans and machines

Python Is Often Favored — But Not Universally

FINDING 5

Python has emerged as a leading candidate for introductory programming, but the research reveals nuance rather than consensus.

The Case for Python

Asghar, Habib, Hussain, and Hussain (2025) conclude that Python produces better learning outcomes and engagement than C++ in many introductory settings. Its advantages include simplicity, readability, and syntactic regularity — qualities that reduce the barrier to initial understanding.

The Ongoing Debate

Cho, Farooq, and Anwar show that educators still disagree. Python, C++, and MATLAB all continue to appear in first-language debates depending on program goals, disciplinary traditions, and institutional context. There is no single winner across all settings.

Key Tension

Python's popularity reflects genuine pedagogical strengths, but context determines whether those strengths translate to a given course. Program goals, student backgrounds, and career pathways all shape the right choice.

Some Researchers Recommend Multiple Languages Instead of One

FINDING 6

A provocative alternative to the "one best language" question is to ask whether the question itself is too narrow. Zampirolli and colleagues propose an introductory course that lets students choose among multiple programming languages, reporting remarkably strong outcomes.

90%

Average Pass Rate

Achieved across remote classes
using a multi-language introductory
course model

3+

Languages Offered

Students could select from multiple
languages based on interest and
goals

1

Shared Goal

All language tracks converged on the
same core computational thinking
outcomes

This approach reframes the problem: rather than optimizing the language, optimize learner agency and motivation. When students have ownership of their tool, engagement and persistence increase.

Personal Affinity Also Influences Learning

FINDING 7

The Understudied Factor

Costa Neto, Araújo, Varanda Pereira, and Henriques argue that affinity toward a programming language is an important factor that has been under-studied relative to technical language difficulty. A learner's subjective connection to a language — its aesthetics, community, or perceived relevance — may shape persistence and performance as much as objective complexity. This finding challenges purely technical evaluations of language suitability and suggests that learner psychology and identity deserve a larger role in curriculum design decisions.



i Affinity is not the same as ease. A language a student *likes* may still be challenging — but that challenge becomes productive rather than discouraging when motivation is high.

Conclusion: The Broader Principle

The research does not support a single best first programming language for every learner. Instead, it supports a broader design principle: the best choice is the one that minimizes friction, fits the course design, and helps beginners focus on problem-solving rather than syntax overhead.

1

Pedagogy First

Choose languages based on learning goals, not industry trends

2

Reduce Cognitive Load

Use visual tools, clean syntax, and fast feedback to support novices

3

Embrace Flexibility

Consider multi-language approaches and learner affinity

4

Context Is King

The right language depends on program goals, student background, and teaching tradition

The strongest pattern across the literature is that beginner success improves when the language and learning environment reduce syntax friction, lower cognitive load, support fast feedback, and fit the learner's context.

Compiled by Pranjal Raj Tomar · Sources: Kölling (2024), Bau et al., Zavgorodniaia et al. (2020), Taylor (1977), Kölling et al. (2017), Winkler & Flatscher, Asghar et al. (2025), Cho et al., Zampirolli et al. (2021), Costa Neto et al.